

CSE33 I: Microprocessor Interfacing and Embedded Systems

Lecture#10: Interrupt Part#1 (Chapter#11)

Interrupts

❓ Motivations

- ❓ Inform a program of some external events timely
 - ❓ Polling vs Interrupt
- ❓ Implement multi-tasking with priority support

Merriam-Webster:

“to break the uniformity or continuity of”

Polling vs Interrupt



www.Vecto.rs · 22705

Polling:

You **pick up the phone every three seconds** to check whether you are getting a call.

Interrupt:

Do whatever you should do and pick up the phone **when it rings**.

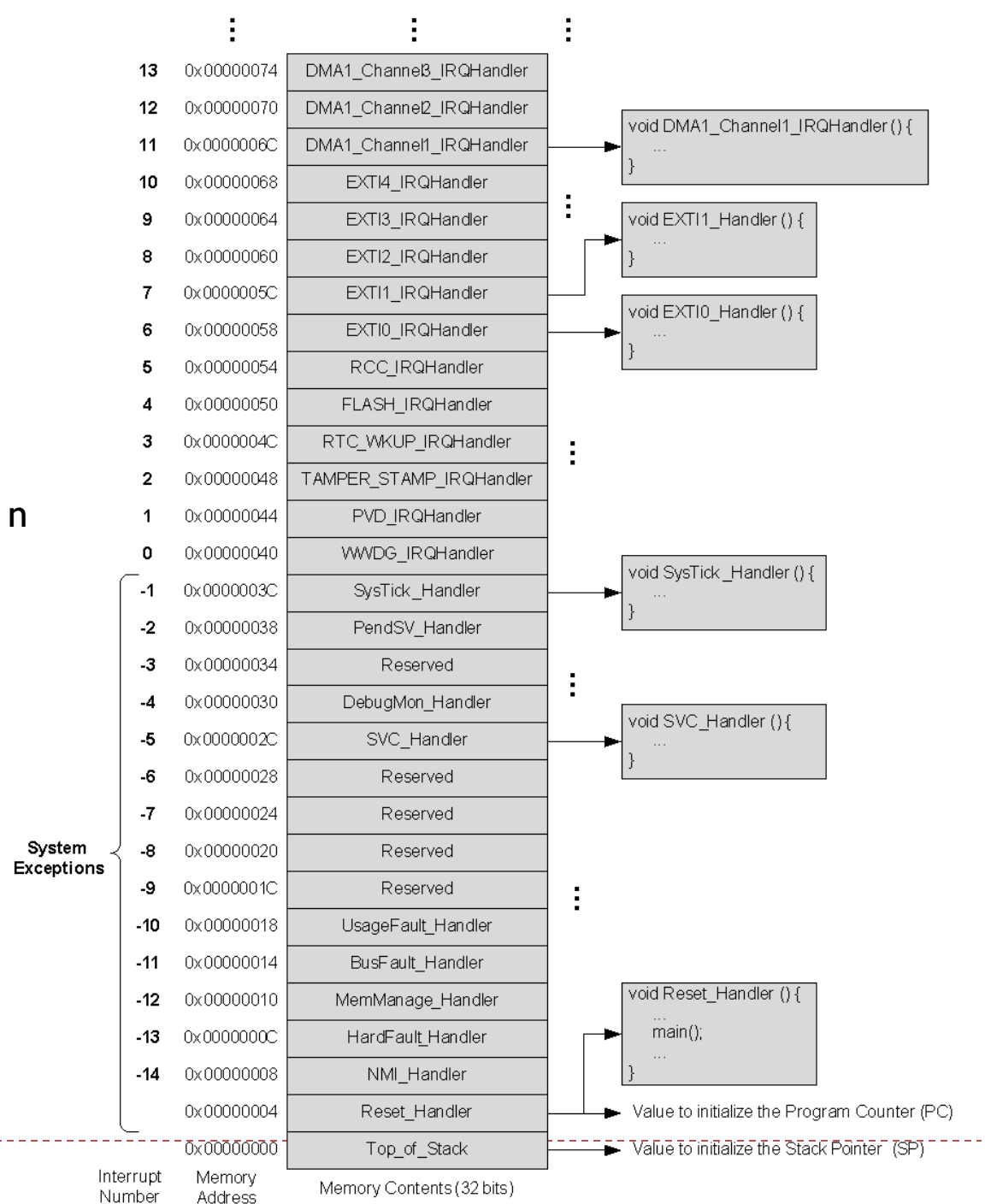
Interrupt Service Routine Vector Table

- ? Start address for the exception handler for each exception type is fixed and pre-defined
- ? Processor loads PC with this fixed, pre-defined address
- ? Exception Vector Table starts at memory address 0
- ? Program Counter **pc** = **0x00000004** initially

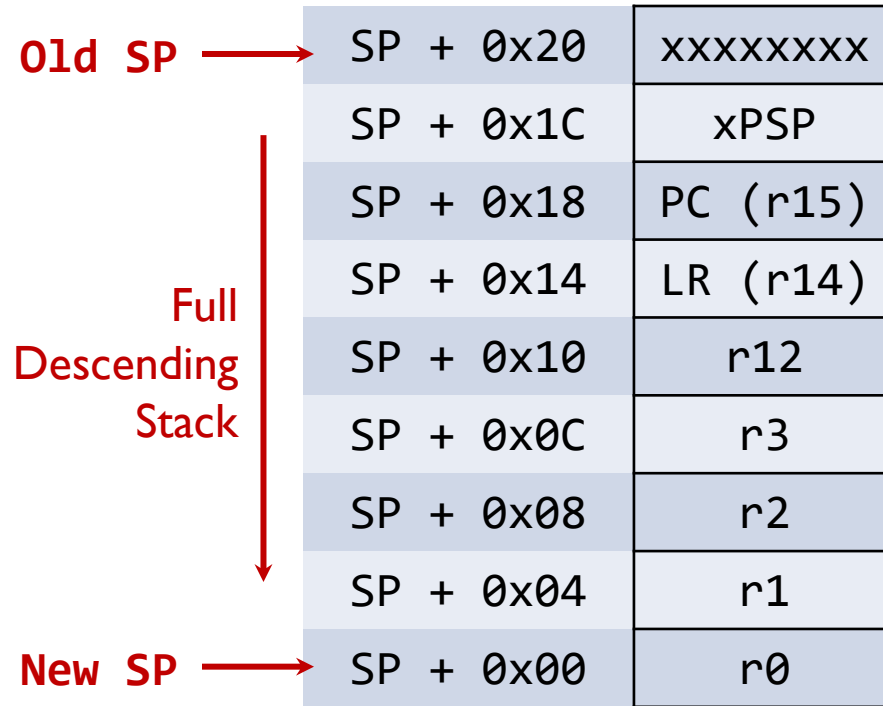
Address	Priority	Type of priority	Acronym	Description
0x0000_0000	-	-	-	Stack Pointer
0x0000_0004	-3	fixed	Reset	Reset Vector
0x0000_0008	-2	fixed	NMI_Handler	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.
0x0000_000C	-1	fixed	HardFault_Handler	All class of fault
0x0000_0010	0	settable	MemManage_Handler	Memory management
0x0000_0014	1	settable	BusFault_Handler	Pre-fetch fault, memory access fault
0x0000_0018	2	settable	UsageFault_Handler	Undefined instruction or illegal state
0x0000_001C-0x0000_002B	-	-	-	Reserved
0x0000_002C	3	settable	SVC_Handler	System service call via SWI instruction
0x0000_0030	4	settable	DebugMon_Handler	Debug Monitor
0x0000_0034	-	-	-	Reserved
0x0000_0038	5	settable	PendSV_Handler	Pendable request for system service
0x0000_003C	6	settable	SysTick_Handler	System tick timer
...				

ISR Vector Table

For interrupt number n:
CMSIS Interrupt Number = 16 + n
 Common Microcontroller Software
 Interface Standard (**CMSIS**)



Stacking & Unstacking



- **Stacking:** The processor automatically pushes these eight registers into the main stack before an interrupt handler starts
- **Unstacking:** The processor automatically pops these eight registers out of the main stack when an interrupt handler exits.

- ? Two stack pointers: Main SP (MSP) and Process SP (PSP)
- ? Determined by operating mode, and bit 0 of the CONTROL register
 - ? Handler mode $\rightarrow SP = MSP$
 - ? Thread mode $\rightarrow SP = MSP$, if $CONTROL[0] = 0$ (i.e., privileged thread mode);
 $SP = PSP$, if $CONTROL[0] = 1$ (i.e., unprivileged thread mode)

MSP vs PSP

In ARM processors, PSP (Process Stack Pointer) and MSP (Main Stack Pointer) are two important concepts related to handling stack operations and managing resources in the processor.

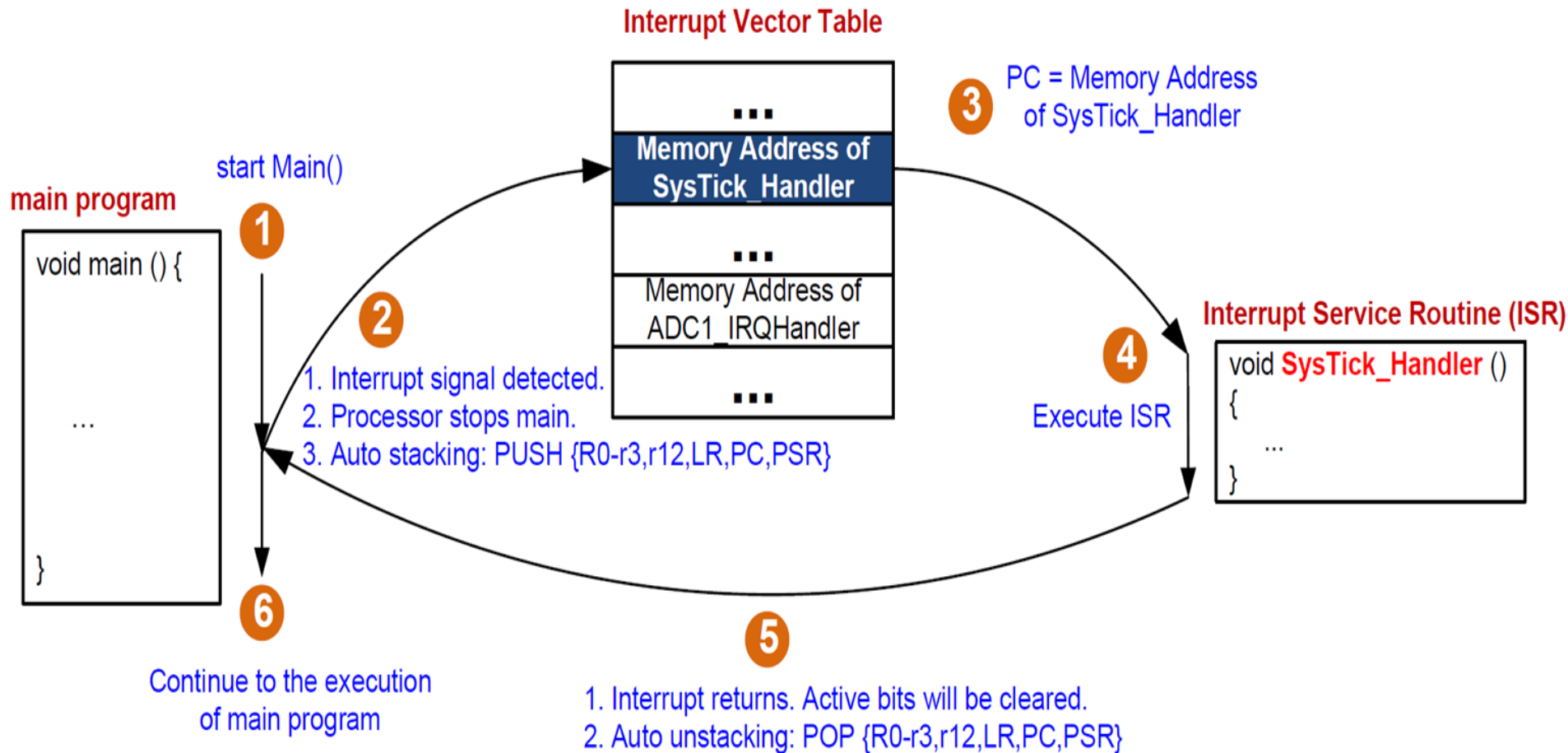
1. MSP (Main Stack Pointer):

- The MSP is the stack pointer used in Handler mode.
- It points to the main stack used in Handler mode.
- Handler mode is a privileged mode in ARM processors primarily used for handling exceptions and interrupts.
- The main stack is typically used for storing context information when an exception or interrupt occurs.

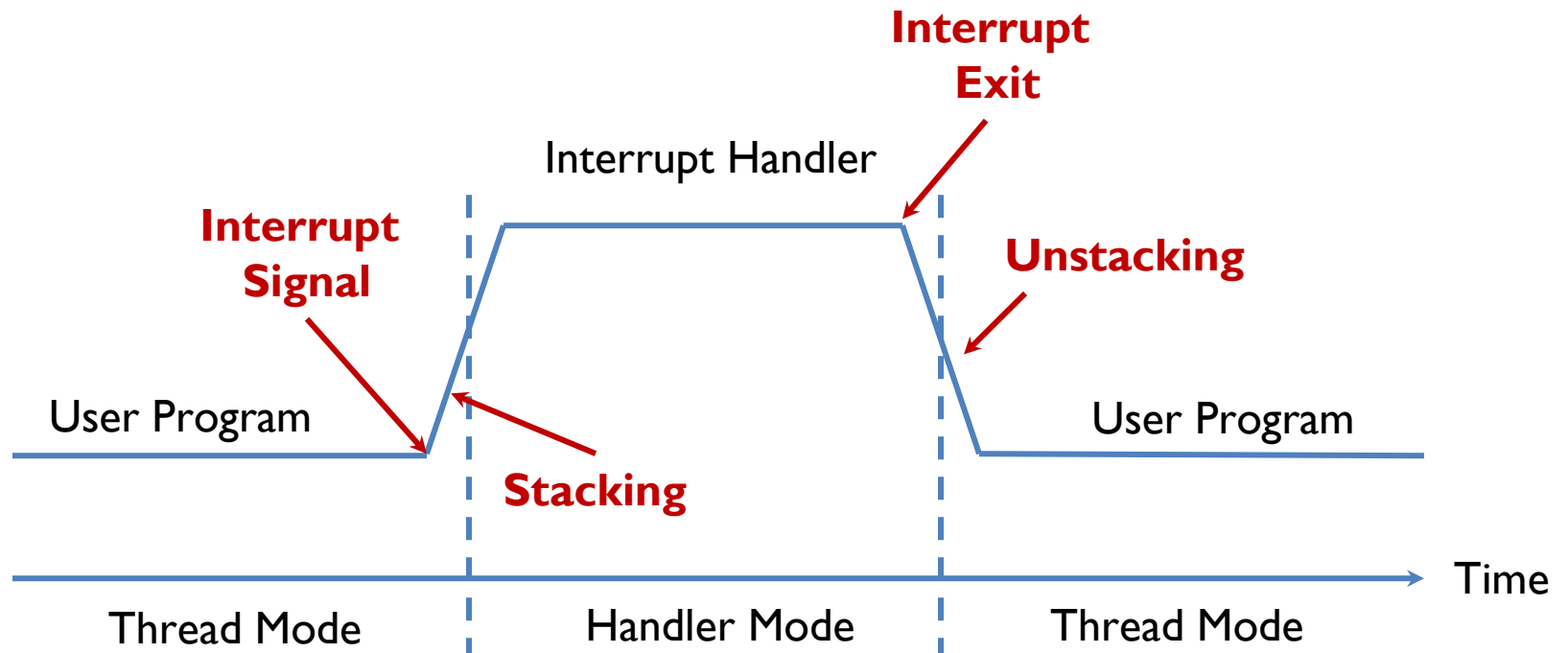
2. PSP (Process Stack Pointer):

- The PSP is the stack pointer used in Thread mode.
- It points to the process stack used in Thread mode.
- Thread mode is an unprivileged mode used for executing application code.
- The process stack is used for storing context information when the processor switches between different threads or tasks.

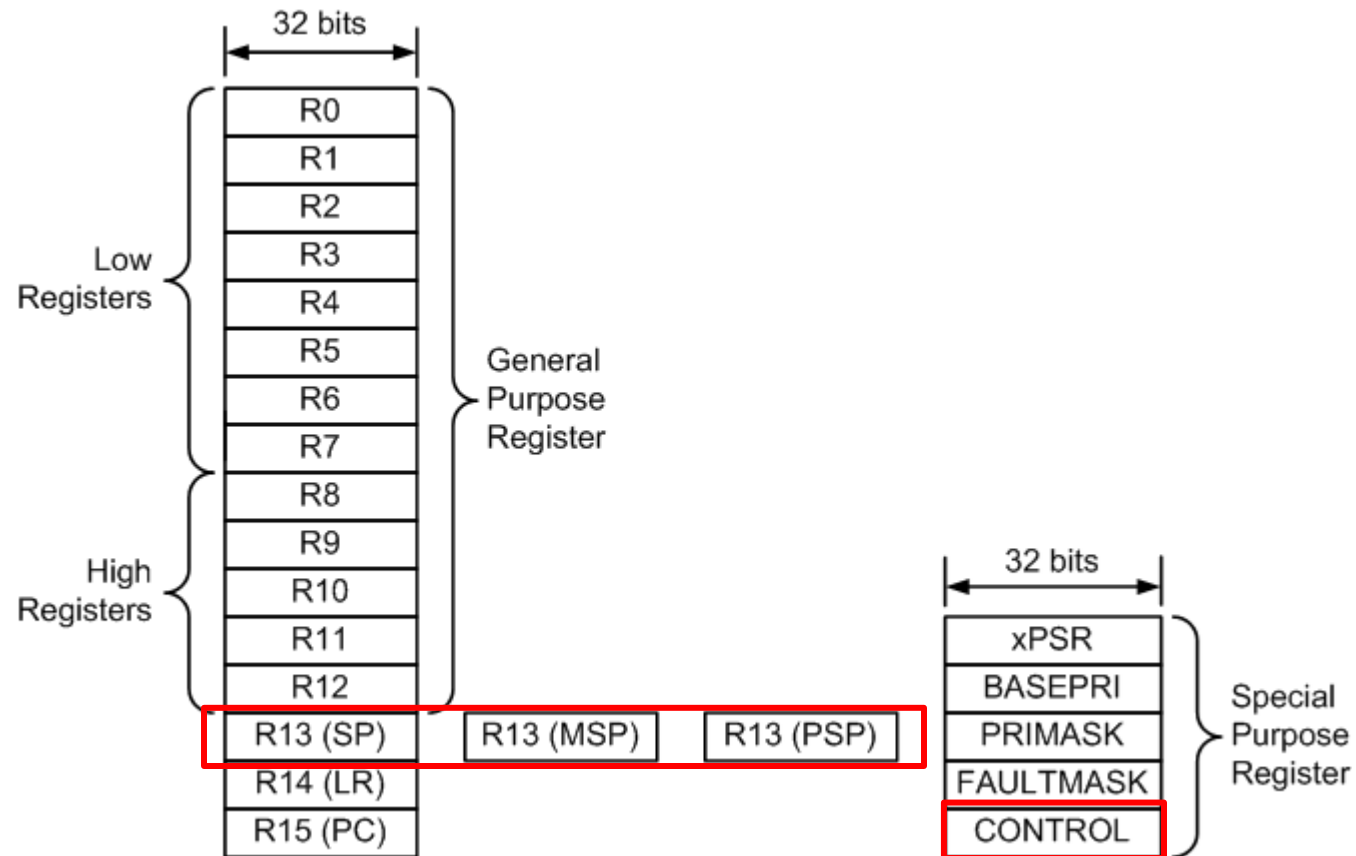
Interrupt



Stacking & Unstacking



Registers



MSP: Main Stack Pointer

PSP: Process Stack Pointer

Processor Mode: Handler Mode *vs* Thread Mode

❑ Handler mode and Thread mode

❑ Handler mode always use **MSP** (Main Stack Pointer)

❑ Thread Mode uses either **PSP** (Process Stack Pointer) or **MSP**

❑ $\text{Control}[1] = 0$, $\text{SP} = \text{MSP}$ (default)

❑ $\text{Control}[1] = 1$, $\text{SP} = \text{PSP}$

❑ When the processor is reset, the default is the thread mode.

❑ The processor enters the handler mode when an exception occurs.



Register values in a interrupt service routine

LR = 0xFFFFFFFF9

SP = MSP

ISR always in
handler mode.

The screenshot shows a debugger interface with the following components:

- Registers Window:** Displays the state of various registers. The 'Core' section shows R0-R12 with values like 0x080001A3 and 0x200005F8. R13 (SP) is 0x200005C8, R14 (LR) is 0xFFFFFFFF9, and R15 (PC) is 0x0800017C. The 'Banked' section shows MSP at 0x200005C8 and PSP at 0x00000000. The 'System' section shows BASEPRI, PRIMASK, FAULTMASK, and CONTROL all at 0x00. The 'Internal' section shows Mode as 'Handler', Privilege as 'Privileged', Stack as 'MSP', States as 2740, and Sec as 0.00034250.
- Disassembly Window:** Shows assembly code for the 'SVC_Handler' procedure. The current instruction at address 0x0800017C is 'CPSID I'. The code includes stack frame setup, saving registers, and a loop that saves the stack pointer and program counter.
- Logic Analyzer Window:** Shows the same assembly code for the 'SVC_Handler' procedure, with a breakpoint set at the 'CPSID I' instruction.

Red arrows point from the text annotations to the corresponding register values in the debugger:

- From 'LR = 0xFFFFFFFF9' to R14 (LR) in the Registers window.
- From 'SP = MSP' to MSP in the Banked registers section.
- From 'ISR always in handler mode.' to the 'Mode' field in the Internal registers section.



Which stack to use when exiting an interrupt?

Link Register (LR) now has two usages:

- ❑ LR = address of the instruction immediately after BL
- ❑ LR indicates whether MSP or PSP is used to restore register from when exiting an interrupt
 - ❑ LR value generated by processor
 - ❑ The processor set LR to **0xFFFFFFFF** on reset.
 - ❑ If return to handler mode, the MSP stack is always used

Which stack to use when exiting an interrupt?

Link Register (LR) now has two usages:

- ❓ LR = address of the instruction immediately after BL
- ❓ LR indicates whether MSP or PSP is used to restore register from when exiting an interrupt

No FP extension:

Link Register (LR)	Return Mode	Return Stack
0xFFFFFFFF1	Handler	SP = MSP
0xFFFFFFFF9	Thread	SP = MSP
0xFFFFFFFDD	Thread	SP = PSP

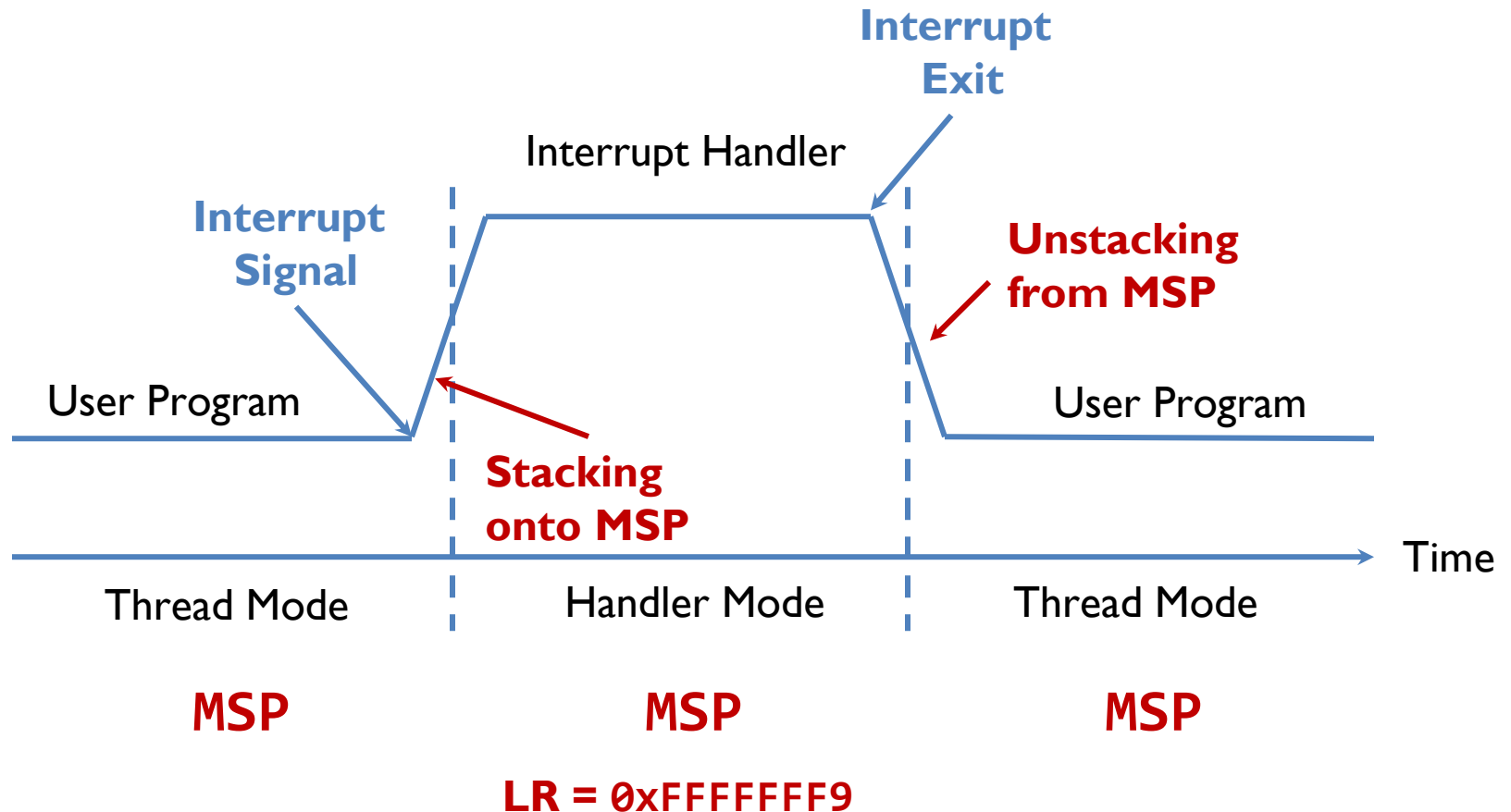
With FP extension:

Link Register (LR)	Return Mode	Return Stack
0xFFFFFFF1E1	Handler	SP = MSP
0xFFFFFFF1E9	Thread	SP = MSP
0xFFFFFFF1ED	Thread	SP = PSP



Stacking & Unstacking

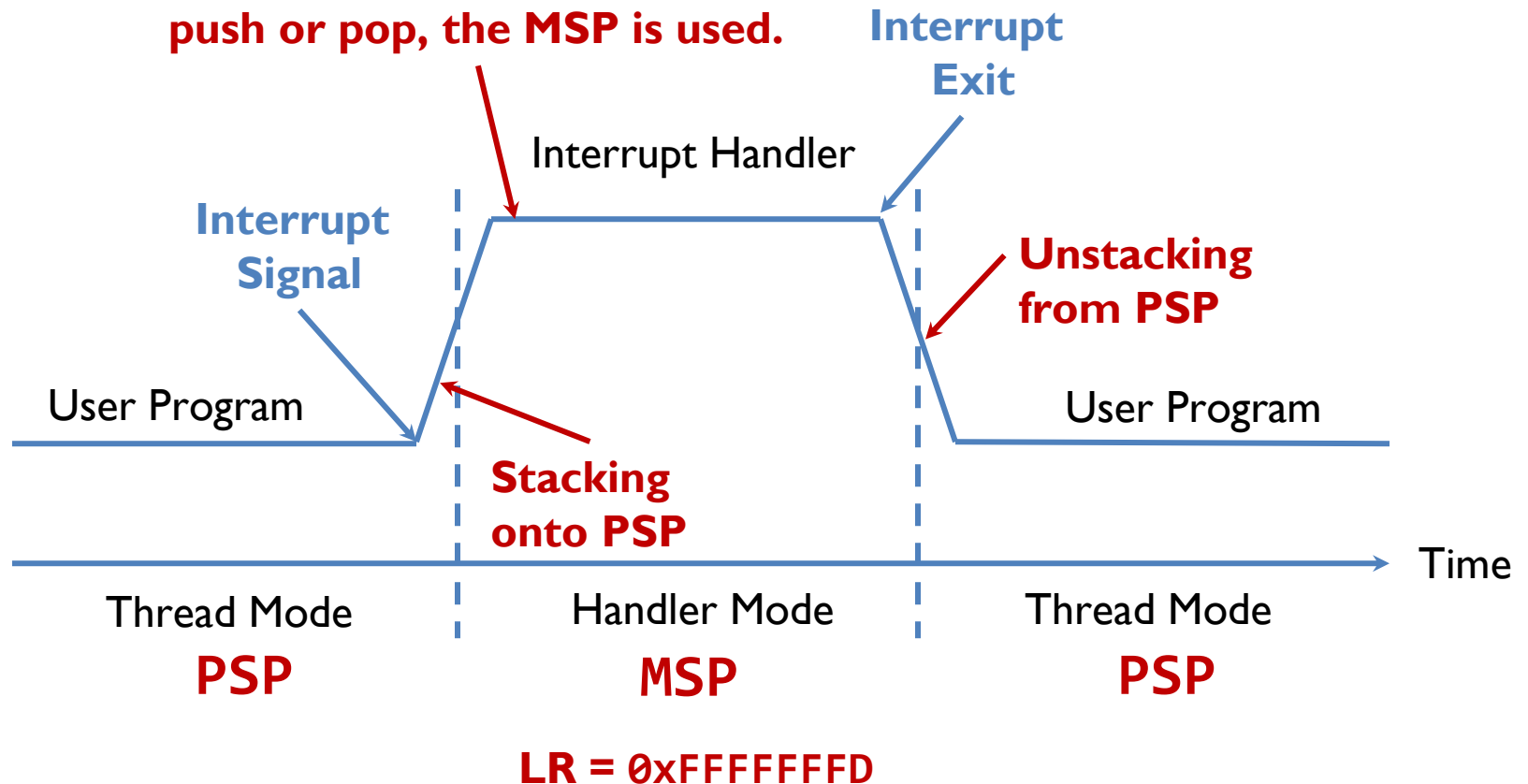
Control[1] = 0 $\Rightarrow$ User program uses MSP.



Stacking & Unstacking

Control[1] = 1 $\Rightarrow$ User program uses PSP.

**If the interrupt handler calls
push or pop, the MSP is used.**



Which is being used?

When exiting an interrupt:

- ? If **LR = 0xFFFFFFFF9**, then **SP = MSP**
- ? If **LR = 0xFFFFFDD**, then **SP = PSP**

9 = 1001

D = 1101

```
TST    r7, #0x04
MRSEQ  r4, msp //Move to Register from Special Register
MRSNE  r4, psp
STMFD  r4!, {r7-r9} ; Push onto a Full Descending Stack
```

Store Multiple Decrement Before (Store Multiple Full Descending) stores multiple registers to sequential memory locations using an address from a base register.



Interrupt: Stacking & Unstacking

```
__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

R0	0	xxxxxxxx	0x20000200
R1	1		0x200001FC
R2	2		0x200001F8
R3	3		0x200001F4
R4	4		0x200001F0
R12	12		0x200001EC
R13(SP)	MSP		0x200001E8
R14(LR)	0x08001000		0x200001E4
R15(PC)	0x08000044		0x200001E0
xPSR	0x21000000		0x200001DC
MSP	0x20000200		0x200001D8
PSP	0x00000000		0x200001D4
			0x200001D0
			0x200001CF

Memory



Interrupt:

Suppose SysTick interrupt occurs when PC = 0x08000044

```
__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0x08001000
R15(PC)	0x08000044

xPSR	0x21000000
MSP	0x20000200
PSP	0x00000000

xxxxxxxx	0x20000200
	0x200001FC
	0x200001F8
	0x200001F4
	0x200001F0
	0x200001EC
	0x200001E8
	0x200001E4
	0x200001E0
	0x200001DC
	0x200001D8
	0x200001D4
	0x200001D0
	0x200001CF

Memory



Interrupt: Stacking & Unstacking

STACKING

```
__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

R0	0	xxxxxxx	0x20000200
R1	1	xPSR	0x21000000 0x200001FC
R2	2	PC	0x08000044 0x200001F8
R3	3	LR	0x08001000 0x200001F4
R4	4	R12	12 0x200001F0
R12	12	R3	3 0x200001EC
R13(SP)	MSP	R2	2 0x200001E8
R14(LR)	0xFFFFFFFF9	R1	1 0x200001E4
R15(PC)	0x0800001C	R0	0 0x200001E0
			0x200001DC
xPSR	0x21000000		0x200001D8
MSP	0x200001E0		0x200001D4
PSP	0x00000000		0x200001D0
			0x200001CF

Memory



Interrupt: Stacking & Unstacking

STACKING

```
_main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

LR = 0xFFFFFFFF9 to indicate MSP is used.

R0	0	xxxxxxx	0x20000200
R1	1	xPSR 0x21000000	0x200001FC
R2	2	PC 0x08000044	0x200001F8
R3	3	LR 0x08001000	0x200001F4
R4	4	R12 12	0x200001F0
R12	12	R3 3	0x200001EC
R13(SP)	MSP	R2 2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1 1	0x200001E4
R15(PC)	0x0800001C	R0 0	0x200001E0
			0x200001DC
xPSR	0x21000000		0x200001D8
MSP	0x200001E0		0x200001D4
PSP	0x00000000		0x200001D0
			0x200001CF

Memory

Interrupt: Stacking & Unstacking

```
__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

R0	0	xPSR	xxxxxxx	0x20000200
R1	1		0x21000000	0x200001FC
R2	2	PC	0x08000044	0x200001F8
R3	4	LR	0x08001000	0x200001F4
R4	4	R12	12	0x200001F0
R12	12	R3	3	0x200001EC
R13(SP)	MSP	R2	2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1	1	0x200001E4
R15(PC)	0x0800001C	R0	0	0x200001E0
				0x200001DC
xPSR	0x21000000			0x200001D8
MSP	0x200001E0			0x200001D4
PSP	0x00000000			0x200001D0
				0x200001CF

Memory

Interrupt: Stacking & Unstacking

```
__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

R0	0	xPSR	xxxxxxx	0x20000200
R1	1		0x21000000	0x200001FC
R2	2	PC	0x08000044	0x200001F8
R3	4	LR	0x08001000	0x200001F4
R4	5	R12	12	0x200001F0
R12	12	R3	3	0x200001EC
R13(SP)	MSP	R2	2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1	1	0x200001E4
R15(PC)	0x08000020	R0	0	0x200001E0
				0x200001DC
xPSR	0x21000000			0x200001D8
MSP	0x200001E0			0x200001D4
PSP	0x00000000			0x200001D0
				0x200001CF

Memory



Interrupt: Stacking & Unstacking

```
_main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

LR = 0xFFFFFFFF9 to indicate MSP is used.

R0	0	xPSR	xxxxxxx	0x20000200
R1	1		0x21000000	0x200001FC
R2	2	PC	0x08000044	0x200001F8
R3	4	LR	0x08001000	0x200001F4
R4	5	R12	12	0x200001F0
R12	12	R3	3	0x200001EC
R13(SP)	MSP	R2	2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1	1	0x200001E4
R15(PC)	0x08000024	R0	0	0x200001E0
				0x200001DC
				0x200001D8
				0x200001D4
				0x200001D0
				0x200001CF

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

Memory

Interrupt: Stacking & Unstacking

UNSTACKING

```
_main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

LR = 0xFFFFFFFF9 to indicate MSP is used.

R0	0	xxxxxxx	0x20000200
R1	1	xPSR	0x21000000
R2	2	PC	0x08000044
R3	4	LR	0x08001000
R4	5	R12	12
R12	12	R3	3
R13(SP)	MSP	R2	2
R14(LR)	0xFFFFFFFF9	R1	1
R15(PC)	0x08000024	R0	0
xPSR	0x21000000		
MSP	0x200001E0		
PSP	0x00000000		

Memory



Interrupt: Stacking & Unstacking

UNSTACKING

```
__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

**Note the new value
of R3 is lost!!!**

R0	0	xxxxxxx	0x20000200
R1	1		0x200001FC
R2	2		0x200001F8
R3	3		0x200001F4
R4	5		0x200001F0
R12	12		0x200001EC
R13(SP)	MSP		0x200001E8
R14(LR)	0x08001000		0x200001E4
R15(PC)	0x08000044		0x200001E0
xPSR	0x21000000		0x200001DC
MSP	0x20000200		0x200001D8
PSP	0x00000000		0x200001D4
			0x200001D0
			0x200001CF

Memory



Interrupt: Stacking & Unstacking

```
__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

addr = 0x08000044

addr = 0x0800001C

The Main program resumes!!!

R0	0	xxxxxxxx	0x20000200
R1	1		0x200001FC
R2	2		0x200001F8
R3	3		0x200001F4
R4	5		0x200001F0
R12	12		0x200001EC
R13(SP)	MSP		0x200001E8
R14(LR)	0x08001000		0x200001E4
R15(PC)	0x08000044		0x200001E0
xPSR	0x21000000		0x200001DC
MSP	0x20000200		0x200001D8
PSP	0x00000000		0x200001D4
			0x200001D0
			0x200001CF

Memory

